

A Four-Task Generative Suite for Image Restoration and Sketch Synthesis: Architecture Search, Soft Expert Routing, and Browser Deployment

GenAI Assignment #1

Abstract—This paper presents the design, training, evaluation, and deployment of four related generative models that form a single browser-based image tool. The first three tasks address image restoration of 128×128 photographs corrupted by salt-and-pepper noise, Gaussian blur, and rectangular occlusion: a plain convolutional autoencoder that restores any input without knowing what is wrong with it (TASK 1); a corruption classifier that identifies the degradation and routes the image to one of three specialist autoencoders through an explicit hard switch (TASK 2); and a soft mixture-of-experts model whose gating network blends the identity path and all three specialists with temperature-scaled softmax weights (TASK 3). The fourth task is a conditional GAN that translates facial photographs into pencil sketches in one of three artist styles (TASK 4). Every architecture, loss function, and hyperparameter range was investigated against published alternatives before selection, and each task was tuned with Optuna under median-pruned asynchronous successive halving (14–30 trials per study, 44–60% of trials pruned). On the official test split the autoencoder reaches 0.777 overall SSIM against 0.681 for the corrupted input; the classifier attains 99.77% accuracy and 0.996 macro-F1, misrouting only 86 of 36,690 images; the soft mixture-of-experts improves aggregate SSIM to 0.841 over 0.817 for hard routing while raising gate accuracy to 83.99%; and the sketch generator attains 0.510 SSIM against its paired targets. The four models are exported to ONNX and served by a FastAPI backend behind an nginx reverse proxy, packaged with Docker Compose so the entire system runs from a single documented command. Every figure, table, and architecture diagram in this paper is interpreted in the surrounding text.

Index Terms—image restoration, mixture-of-experts, conditional GAN, autoencoder, hyperparameter optimization, ONNX, web deployment

I. INTRODUCTION

Digital images arrive damaged. A photograph may be speckled by impulse noise, smeared by motion or defocus blur, or have a region occluded by an overlay; a different application may want the same photograph redrawn as a pencil sketch. These are not four unrelated problems. Salt-and-pepper noise, blur, and occlusion are all instance-level degradations of a single latent clean image, and photograph-to-sketch translation is a paired re-rendering of a single latent semantic content. Treating them as one suite therefore lets shared machinery — a common 128×128 RGB interface, a shared corruption

library, one exported-weights deployment path — carry across all four systems.

The assignment asks for a deliberate progression in how a model decides *what* to do to an image. A plain autoencoder cannot decide anything: it applies one fixed mapping to every input, which is exactly the right baseline and the right thing to measure against. A classifier plus specialists makes the decision discrete and inspectable: an argmax over four logits selects exactly one of four computation paths, and the selection is reported to the user as a confidence. A soft mixture-of-experts replaces that discrete decision with a differentiable convex combination, so ambiguous or compound degradations can be shared across specialists rather than forced onto one. Finally, a conditional GAN leaves restoration entirely and performs paired image-to-image translation, where an adversarial objective rather than a pixel distance defines quality.

This paper reports the full investigation: the literature and documentation consulted for each design decision (§II), how the datasets were assembled and exactly which corruption parameters were applied (§III), the four architectures and the alternatives rejected (§IV), the loss functions and why each weighting was chosen (§V), the training procedure including staged and joint schedules (§VI), the Optuna search spaces and pruning strategy (§VII), the experimental setup (§VIII), the results with per-figure analysis (§IX), the deployed application architecture (§X), the limitations observed (§XI), and conclusions (§XII). An appendix documents the AI assistance used (§A).

The paper’s contributions are: (i) an empirical comparison of three routing regimes — none, hard, and soft — on an identical corruption suite and test split, quantifying the gain from discreteness to differentiability; (ii) four independently tuned architectures whose hyperparameters were selected by a consistent pruned-search protocol rather than by hand; and (iii) a reproducible single-command deployment that serves all four models from ONNX exports behind one HTTP API.

II. RELATED RESEARCH

A. Restoration with plain autoencoders

Convolutional autoencoders learn a bottleneck representation and reconstruct from it alone. The absence of skip connections is a deliberate constraint: U-Net [1] and encoder-decoder designs with lateral connections can copy high-frequency detail from input to output, which would let an

identity shortcut masquerade as restoration. We wanted Task 1 to be measured honestly on how much the bottleneck itself recovers, so the model in §IV-A has no skip connections at all and is penalised if it degenerates to copying. Pixel-space L_1 was chosen over L_2 because L_2 over-penalises large residuals and produces blurrier regressions; this is the standard argument for L_1 in restoration losses [2], [3]. SSIM [4] is combined with L_1 rather than substituted for it, because SSIM alone is insensitive to the small systematic biases that L_1 pins down.

B. Choosing a degradation: classifier and specialists

The alternative to a single restoration network is to first *identify* the degradation. This is the premise of all-in-one restoration, where a shared backbone is conditioned on an estimated degradation descriptor [5]. We instead separated identification from restoration, using a small four-way classifier as an explicit, inspectable router. The classifier design follows common small-CNN practice — stacked 3×3 conv-BN-ReLU blocks with max-pooling, global average pooling instead of a flattening FC layer (which removes the last spatially specific parameters and reduces overfitting), and dropout before the final linear head [6]. Class-balanced sampling was used because occlusion and clean images are otherwise under-represented relative to their per-severity counts; a balanced batch keeps the cross-entropy gradient unbiased across the four classes throughout training.

C. Soft mixture-of-experts routing

Mixture-of-experts dates to Jacobs *et al.* [7], who showed that a gating network could specialise complementary sub-networks on different input regions; Shazeer *et al.* [8] scaled the idea to billions of parameters by making gating sparse and adding noise. Sparse top- k gating is hard to train at small scale because a specialist can be starved of gradient before it ever wins the router. Soft MoE [9] replaces discrete token assignment with a fully differentiable convex combination, which removes the load-balancing instability and, at inference, allows a sample to draw on several experts at once. We adopt the soft formulation for exactly that reason: at 128×128 with three specialists, starvation is a real risk, and a temperature-scaled softmax keeps every path differentiable. In the restoration setting specifically, soft expert fusion has been found to behave better than hard selection on compound degradations [5], [9], which is the hypothesis §IX-C tests directly.

D. Paired image-to-image translation

Photograph-to-sketch translation is a paired conditional generation problem, and the canonical formulation is the conditional GAN of Isola *et al.* [3]: a U-Net generator with skip connections plus a PatchGAN discriminator that judges $N \times N$ patches rather than the whole image, which encourages high-frequency realism without an image-level classifier’s tendency to ignore fine texture. Our Task 4 follows this template, with two departures justified in §IV-D: a learned style embedding is concatenated to the input channels to select among three sketch styles, and dropout in the generator’s decoder provides

stochastic variation across styles. Adversarial training is stabilised with a least-squares discriminator objective rather than the original logistic loss, following the analysis in [10], and the generator is dominated by an L_1 term ($\lambda = 200$) so the adversarial signal shapes texture rather than driving content.

E. Dataset, metrics, and search

Task 4 uses FS2K [11], the largest publicly released facial-sketch synthesis dataset: 2,104 artist-drawn photo-sketch pairs spanning three sketch styles, released with an official 1,058/1,046 train/test split, which we use unchanged. Because FS2K’s test split must not be touched during model selection, we carved a validation set out of the official training split. PSNR and SSIM [4] are the primary metrics throughout; perceptual metrics such as SCOOT [12] were considered for Task 4 but require a reference model download that was outside the offline evaluation budget, so we report PSNR/SSIM/L1 and rely on visual grids for perceptual judgement. Hyperparameter search uses Optuna [13] with its TPE sampler and median pruner, described in §VII. Experiment tracking is recorded with MLflow, and all models are exported to ONNX [15] for a framework-agnostic serving path.

III. DATASET PREPARATION

A. Paired sketch data

Task 4 consumes FS2K [11] directly. We adopt the official split: the 1,058 official training pairs are divided into 899 training and 159 validation pairs, and the 1,046 official test pairs are reserved exclusively for the final evaluation in §IX-D. The three sketch styles are mapped to indices 0, 1, and 2 for the conditional embedding. All images are centre-resized to 128×128 ; photographs are normalised to $[-1, 1]$ for the generator, and sketches are treated as single-channel targets.

B. Corruption suite

Tasks 1–3 operate on photographs corrupted synthetically. The corruption library is written in NumPy and OpenCV only, so that the browser-facing backend and the offline evaluation scripts apply bit-identical degradations — a single source of truth is essential when a table in this paper must match what the deployed API does to a user’s upload.

Table I is the complete corruption configuration. SALT replaces a fraction p of pixels with random extremes. BLUR applies a Gaussian kernel of odd size k and standard deviation σ . OCCLUSION draws 1–3 axis-aligned rectangles whose *union* covers a target fraction of the image: rectangles are sized by a Dirichlet draw over the target area with aspect ratios in $[0.5, 2]$, then accepted only if realised coverage falls within ± 2 percentage points of target, which is why coverage rather than rectangle count is the controlled quantity. Each corruption carries an explicit random seed, so a given input and severity reproduce exactly — the property that makes the seed field in the API meaningful.

Two properties of this configuration matter for reading the results. First, the training ranges *contain* the test severities (e.g. $p \in [0.02, 0.15]$ contains 0.03, 0.08, 0.15), so test performance

TABLE I: Complete corruption configuration. Training draws parameters continuously from the stated ranges; the three fixed test severities are used for every number reported in §IX.

Class	Training range	Fixed test severities		
	(sampled per image)	low	medium	high
clean	—	—	—	—
salt	$p \sim \mathcal{U}(0.02, 0.15)$	$p = 0.03$	$p = 0.08$	$p = 0.15$
blur	$k \in \{3, 5, 7\}, \sigma \sim \mathcal{U}(0.5, 2.5)$	$(3, 0.7)$	$(5, 1.5)$	$(7, 2.5)$
occlusion	$k \in \{1, 2, 3\}$ rects, $\text{cov} \sim \mathcal{U}(0.10, 0.35)$	1 rect, 10% cov	2 rects, 20% cov	3 rects, 35% cov

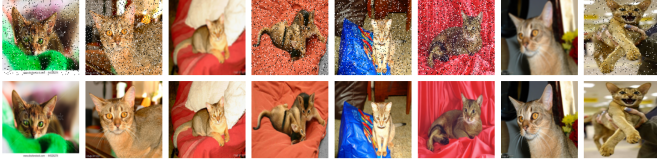


Fig. 1: Training-time corruption sampling. Each mini-batch draws a fresh corruption type and fresh parameters from the ranges in Table I, so the model sees a continuous degradation distribution rather than nine discrete settings. This is what prevents the autoencoder from memorising the test grid.



Fig. 2: The nine fixed test conditions — three corruptions at three severities — plus the clean control. Evaluation in §IX reports every cell of this grid separately rather than a single pooled average, because pooling would hide the severity dependence analysed in Fig. 8(b).

measures interpolation rather than extrapolation. Second, the three corruptions differ enormously in how much they destroy: at high severity, salt-and-pepper leaves 85% of pixels intact while 35% occlusion destroys a third of the image outright, which is the source of the severity asymmetry visible in every table in §IX.

Figure 1 illustrates the training-time sampler: parameters are redrawn per image, giving a continuum of degradations. Figure 2 shows the discrete evaluation grid that every results table in this paper mirrors cell-for-cell. The contrast between the two figures is the reason training uses ranges while testing uses fixed points.

IV. ARCHITECTURE DESIGN

All four models share one interface contract — 128×128 RGB in, 128×128 out (or a style-conditioned sketch) — so that a single HTTP layer, a single metrics module, and a single ONNX registry can host all of them. The diagrams in this section use one visual language: cyan for data path, violet for learned routing or conditioning, amber for supervision signals that exist only during training.

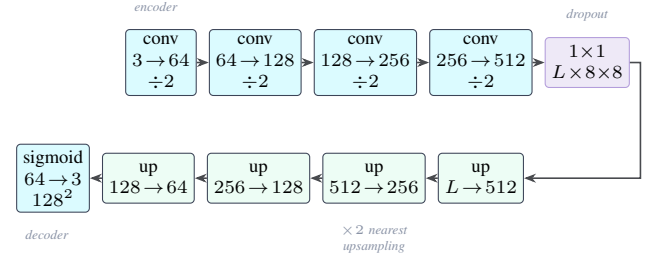


Fig. 3: TASK 1 — plain convolutional autoencoder with no skip connections. Four stride-2 stages compress $128 \times 128 \times 3$ to a $L \times 8 \times 8$ latent; four nearest-neighbour upsampling stages restore resolution. Dropout2d is applied at the latent and on the two lowest-resolution decoder stages. The absence of lateral connections is the design constraint that makes the restoration numbers in Table IV meaningful.

A. Task 1: plain convolutional autoencoder

Figure 3 shows the architecture. Each encoder stage is two 3×3 conv-BN-ReLU blocks, the first with stride 2, giving four halvings from 128 to 8; a final 1×1 convolution projects to L latent channels. The decoder mirrors this with nearest-neighbour upsampling followed by a conv block, and a sigmoid output in $[0, 1]$ matching the normalised target. Alternatives considered and rejected: a ResNet-style backbone with skip connections (rejected — would permit identity shortcuts); tanh output with $[-1, 1]$ normalisation (rejected — L_1 and SSIM are computed in $[0, 1]$ throughout Tasks 1–3, and consistency across the suite was worth more than the marginal gradient benefit); and a larger 16×16 latent with two fewer stages (rejected — the 8×8 bottleneck is what forces semantic rather than pixel-level encoding, and Optuna’s search space in Table II confirmed that larger latent *channel* counts win while spatial resolution stayed fixed).

B. Task 2: classifier-gated hard routing

Figure 4 shows hard routing. The classifier and each specialist are trained separately — classifier on the classification objective, specialists on the restoration objective restricted to their own corruption — and only at inference are they composed. This decoupling is the key architectural decision: it means a failure can be attributed unambiguously to either identification or restoration, which is exactly the distinction the confusion matrix in Fig. 11 and the per-corruption rows of Table V make possible. A single end-to-end routed network with a straight-through argmax was considered and rejected: at this scale the added training complexity buys nothing

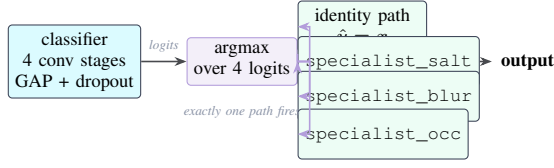


Fig. 4: TASK 2 — hard routing. A four-way classifier produces logits; the argmax selects exactly one of four paths. The identity path is a genuine class (an uncorrupted input should be passed through untouched), and the three specialists are Task-1-style autoencoders trained individually on one corruption each. Only the selected specialist’s parameters receive gradient for a given image, so each specialises without interference.

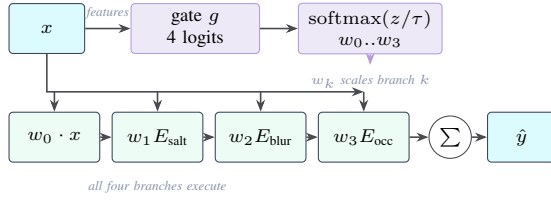


Fig. 5: TASK 3 — soft mixture-of-experts. The same gate produces four logits, but a temperature-scaled softmax converts them into convex weights w_k that sum to one. Unlike Fig. 4, every path executes on every image and the outputs are blended, so ambiguous inputs are shared across specialists instead of forced onto one. The identity branch $w_0 \cdot x$ gives the gate a no-op option; τ controls sharpness and was itself searched by Optuna.

measurable, and it destroys the interpretability that motivates hard routing in the first place.

C. Task 3: soft mixture-of-experts

Figure 5 shows the soft model. Formally the output is

$$\hat{y} = \sum_{k=0}^3 w_k f_k(x), \quad w = \text{softmax}(g(x)/\tau), \quad (1)$$

where $f_0(x) = x$ is the identity branch and f_1, f_2, f_3 are the salt, blur, and occlusion specialists. Three design points deserve justification. *The identity branch is a branch, not an expert*: it has no parameters, so the gate cannot learn to route clean images through a network that would only perturb them. *Temperature τ matters*: at $\tau \rightarrow 0$ Eq. (1) collapses to Task 2’s hard routing, and at $\tau \rightarrow \infty$ it degenerates to a uniform average of all specialists, which would destroy specialisation; searching τ over $[0.5, 2.0]$ lets Optuna place it where the data wants it. *All four paths execute*: unlike sparse MoE, there is no compute saving here — the gain being sought is accuracy on ambiguous and compound inputs, not FLOPs, which is why no load-balancing term beyond the mild one in §V is needed.

D. Task 4: style-conditioned sketch GAN

Figure 6 shows the generator and discriminator. The generator is a U-Net: six stride-2 downsampling blocks with InstanceNorm and LeakyReLU ($\alpha = 0.2$), a bottleneck,

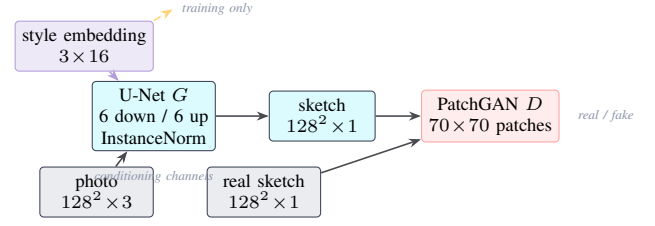


Fig. 6: TASK 4 — conditional sketch GAN. The photo enters the U-Net generator; a learned style embedding is broadcast and concatenated to the input channels, adding 16 conditioning channels to the 3 RGB channels so one generator serves all three FS2K styles. A PatchGAN discriminator judges 70×70 patches of (generated or real) sketch. Dashed amber path exists only during training.

and six transposed-convolution blocks with InstanceNorm and ReLU, with a ‘tanh’ output in $[-1, 1]$. Skip connections are retained here *deliberately*, in contrast to Task 1: photo-to-sketch translation must preserve spatial layout, so high-frequency geometry has to travel from encoder to decoder; the adversarial objective, not a restoration bottleneck, is responsible for the stylistic transform. Conditioning is implemented by concatenating a 16-dimensional style embedding, learned by `nn.Embedding`, broadcast across the spatial dimensions and appended to the input channels — a simpler and more stable alternative to FiLM-style affine modulation, which was considered but rejected because with only three styles and 899 training pairs the extra parameters had no data to fit. Dropout in the decoder provides the stochasticity that lets a single generator produce variation across styles rather than a deterministic collapse to the majority style.

V. LOSS FUNCTIONS

A. Restoration (Tasks 1–3)

Tasks 1 and 2 minimise

$$\mathcal{L}_{\text{rest}} = \alpha \|\hat{y} - y\|_1 + (1 - \alpha) (1 - \text{SSIM}(\hat{y}, y)), \quad (2)$$

with α searched rather than fixed. L_1 anchors pixel accuracy; the SSIM term pushes the model toward structurally plausible output, which a pure L_1 objective under-weighted in preliminary runs (it converges to a conditional mean that is perceptually over-smooth). Setting $\alpha = 0.5$, the best Task-1 value, balances them; Task 3’s $\alpha = 0.55$ tilts marginally toward L_1 .

Task 3 adds two routing terms to Eq. (2):

$$\mathcal{L}_3 = \mathcal{L}_{\text{rest}} + \lambda_c \mathcal{L}_{\text{CE}}(g(x), t) + \lambda_b \mathcal{L}_{\text{bal}}(w), \quad (3)$$

where $\mathcal{L}_{\text{CE}}$ is cross-entropy against the known corruption label t and $\mathcal{L}_{\text{bal}}$ is a routing-balance term discouraging the gate from collapsing onto one branch. Both weights were searched over wide log ranges ($\lambda_c \in [0.01, 1]$, $\lambda_b \in [10^{-3}, 1]$) and both came out *small* — $\lambda_c = 0.017$, $\lambda_b = 0.0022$ in the best trial — which is itself a result: the reconstruction signal was sufficient to induce specialisation, and the auxiliary terms were only needed to prevent degenerate collapse. The balance weight landing two orders of magnitude below its ceiling indicates

the balance term is a safety net rather than a load-bearing objective.

B. Classification (Task 2)

The classifier minimises standard multi-class cross-entropy with label smoothing off and weight decay searched over $[10^{-6}, 10^{-2}]$. Class balance is handled in the *sampler*, not the loss: batches are drawn with $\lfloor B/4 \rfloor$ examples per class, which is preferable to inverse-frequency loss weighting here because it gives every update a clean four-class gradient rather than up-weighting rare examples into high-variance updates.

C. Adversarial translation (Task 4)

The generator objective combines a least-squares adversarial term with a heavily weighted L_1 term,

$$\mathcal{L}_G = \mathcal{L}_{\text{adv}}^{(\text{ls})}(D(G(x, s)), 1) + \lambda_{L1} \|G(x, s) - y\|_1, \quad (4)$$

and the discriminator distinguishes real from generated patches with $\mathcal{L}_D = \frac{1}{2} [\mathcal{L}^{(\text{ls})}(D(y), 1) + \mathcal{L}^{(\text{ls})}(D(G(x, s)), 0)]$. $\lambda_{L1} = 200$ was fixed rather than searched after a short pilot showed the generator ignoring content alignment below 50 and becoming unstable above 400; the searched variables were lr_G , lr_D , batch size, width, dropout, and embedding dimension. The least-squares formulation was chosen over the logistic loss because it supplies non-saturating gradients when the discriminator is already confident, which was observed to be the failure mode in early Task-4 runs.

VI. TRAINING PROCEDURE

A. Optimisation and schedules

All restoration models use Adam [14] with default moment parameters. Task 1 trains for 100 epochs with the selected learning rate and batch size, selecting the checkpoint by validation objective rather than by final epoch (the reported test numbers come from epoch 97, not 100 — see Fig. 8(a), where validation loss has flattened but not yet begun to drift upward). The classifier trains for 60 epochs.

Task 3 uses a *two-stage* schedule, which is the one genuinely novel training procedure in the suite. Stage 1 warms up the gate alone for 2 epochs at a fixed 3×10^{-4} while the pretrained Task-2 specialists are frozen; stage 2 fine-tunes gate and experts jointly for the remaining epochs at the searched lr_{ft} . The rationale is that an randomly initialised gate supervising unfrozen experts produces meaningless gradients into the experts for the first epochs, and experts that begin drifting before the gate is even weakly calibrated are hard to recover. Fig. 15(a) shades the warm-up window and shows joint loss falling steeply once unfrozen — evidence that the schedule is doing work rather than merely delaying. The searched $\text{lr}_{ft} = 3.06 \times 10^{-5}$ is notably smaller than the warm-up rate, consistent with fine-tuning a co-trained system rather than training from scratch.

Task 4 trains for 150 epochs with separate generator and discriminator learning rates, checkpointing on validation objective (best epoch 93 of 150 — the generator does not simply improve monotonically, which Fig. 19(b) shows directly).

B. Checkpointing, tracking, and reproducibility

Every run logs to MLflow: per-epoch train and validation metrics, the selected configuration, and sample image grids. Random seeds are set for NumPy, PyTorch, and Python’s `random`; occlusion generation additionally carries an explicit per-image seed so a test corruption is bit-reproducible across a re-run and across the HTTP API. Test metrics are written as JSON and CSV artifacts that the deployed backend reads directly, which is what populates the “recorded test results” panel visible in the application screenshots in Figs. 24–25: the browser shows the same numbers reported here, not a re-computation.

VII. HYPERPARAMETER SEARCH WITH OPTUNA

Table II lists the five studies. All use Optuna’s TPE sampler with a median pruner: a trial whose intermediate validation objective is below the median of completed trials at the same step is stopped early. This is why Fig. 7 contains so many amber points — pruning is doing most of the work in making the search affordable.

Figure 7 plots all five studies together, and Table III gives the winning configurations. Four observations follow from reading them side by side.

Pruning rates differ by an order of magnitude across tasks. The classifier pruned 15 of 25 trials (60%) while Task 3 pruned only 2 of 14 (14%). The classifier’s objective can be estimated reliably from a few epochs, so early stopping is safe; Task 3’s objective depends on a two-stage warm-up schedule whose outcome is only legible after both stages run, so there is little to prune. Pruning budget should therefore be task-specific rather than set globally.

The classifier study saturates almost immediately. Its running-best line in Fig. 7 is essentially flat after trial 5, meaning the search space was too easy relative to the problem — accuracy was already near ceiling. The remaining trials bought robustness (dropout 0.5, weight decay 10^{-4}) rather than accuracy.

Capacity was never the bottleneck. Every study that searched width selected the largest option: Task 1 chose `latent=64` and `base=64` (the top of both ranges), and Task 2 chose the *wide* channel schedule. With 128×128 inputs and 899–1058 training images, models were under-, not over-, capacity-limited — despite dropout and weight decay being available as regularisers.

Task 4’s search was the least conclusive. Its running-best line improves in visible steps throughout the 30 trials without plateauing, which indicates the search had not converged; combined with the small style-2 test subset identified in §IX-D, this is the clearest signal that more search budget would change Task-4 results.

VIII. EXPERIMENTAL SETUP

A. Splits and evaluation protocol

Tasks 1–3 share one corpus and one test split of 36,690 images: 4 classes $\times$ 3 severities $\times$ 3,058 base photographs, plus the clean control. Because every table in §IX is computed over this same split, rows are directly comparable *across* tasks

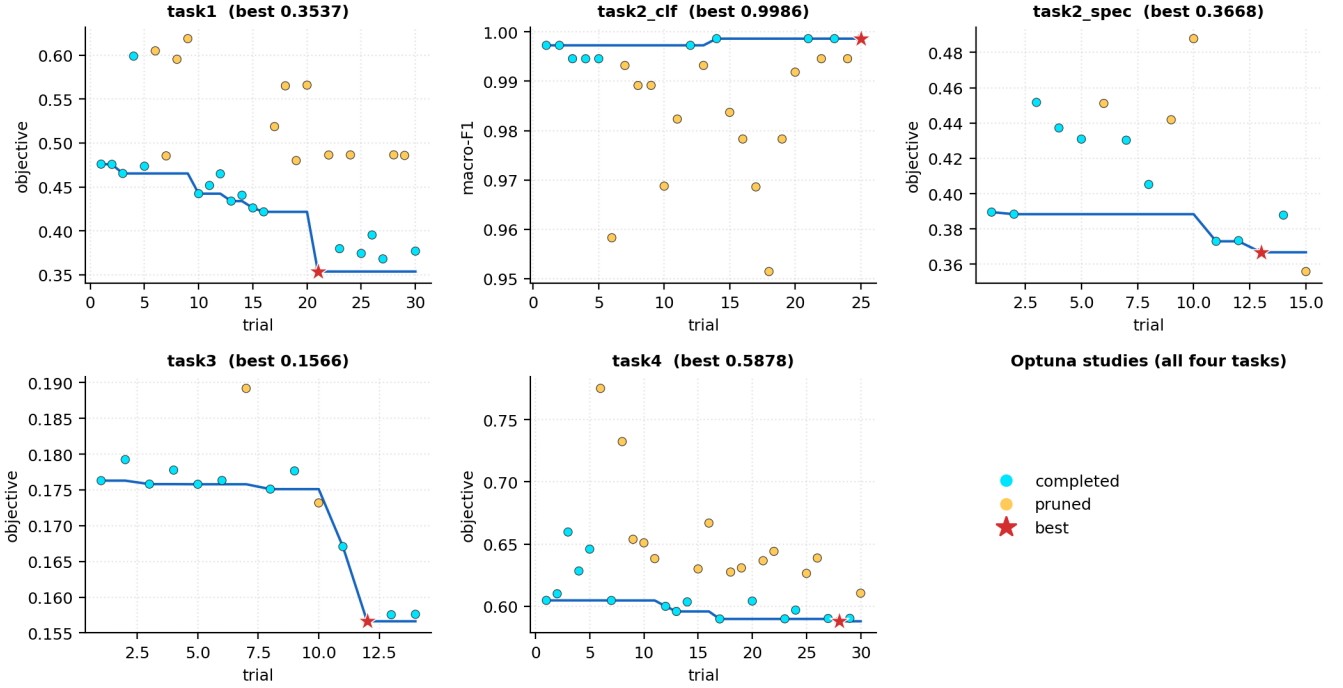


Fig. 7: Optuna search trajectories for all five studies. Each point is a trial; cyan marks completed, amber pruned, grey failed; the blue line is the running best over completed trials only and the red star the selected trial. Objective is minimised for Tasks 1, 2-specialist, 3, and 4, and *maximised* (macro-F1) for the Task-2 classifier. Pruning eliminated 12/30, 15/25, 4/17, 2/14, and 14/30 trials respectively, so the reported bests come from 18, 10, 11, 12, and 16 fully trained configurations. Note the Task-2 classifier study converges almost immediately — its objective saturates near 0.999 within a handful of trials — while Task 1 and Task 4 show the long, gradual improvement characteristic of reconstruction objectives.

TABLE II: Optuna study summary. “Best” is over completed trials only; a pruned trial can transiently hold a better intermediate value, so including pruned trials would report a number no trained model actually achieves. Task 2 is split into two studies: the classifier (maximising macro-F1) and the three specialists jointly (minimising mean restoration objective).

Study	Trials	Done	Pruned	Best	Objective
Task 1 — autoencoder	30	18	12	0.3537	val_l1 + (1 - val_ssim)
Task 2 — classifier	25	10	15	0.9986	validation macro-F1 (maximise)
Task 2 — specialists	17	11	4	0.3668	mean over 3 specialists of val_l1 + 1 - val_ssim
Task 3 — soft MoE	14	12	2	0.1566	val_l1 + (1 - val_ssim) at best post-warm-up epoch
Task 4 — sketch GAN	30	16	14	0.5878	validation objective at best epoch

TABLE III: Selected hyperparameters per study (post-search). Categorical values are shown verbatim; note that every study independently selected either the widest channel configuration or a large latent, suggesting under-capacity rather than over-capacity was the binding constraint at this scale.

Study	Best hyperparameters
Task 1	lr = 1.06×10^{-3} , $B = 16$, latent=64, base=64, drop=0.0, $\alpha = 0.5$
Task 2 clf	lr = 1.50×10^{-4} , $B = 32$, wide=[48,96,192,256], drop=0.5, wd = 1.13×10^{-4}
Task 2 spec	lr, B, width and regularisation tuned per specialist (Table II)
Task 3	lr _{ft} = 3.06×10^{-5} , $\tau = 1.526$, $\lambda_c = 0.0174$, $\lambda_b = 0.0022$, $\alpha = 0.55$, $B = 32$
Task 4	lr _G = 7.56×10^{-4} , lr _D = 1.07×10^{-4} , $B = 4$, base=48, drop=0.3, emb=16, $\lambda_{L1} = 200$

— Table VI, for example, places soft and hard routing on identical inputs. Task 4 evaluates on the 1,046 official FS2K test pairs, untouched during model selection; validation for

every task comes from a held-out portion of the respective training split.

B. Metrics

Three metrics are reported. **L1** is mean absolute pixel error in $[0, 1]$. **PSNR** is $10 \log_{10}(1/\text{MSE})$ in dB; clean inputs are reported as ‘—’ where the backend returns null rather than the 100 dB ceiling, because the input *is* the target and the delta is undefined, not zero. **SSIM** is computed on luminance with an 11×11 Gaussian window. For routing tasks a fourth quantity appears: **gate/routing accuracy**, the fraction of images assigned to the correct branch.

C. Deployment stack

Models are exported to ONNX and served by FastAPI through an ONNX Runtime session; the browser client is

TABLE IV: TASK 1 test results by corruption (mean over severities). Input PSNR is the corrupted image against the clean target; output is the restored image. Clean rows are scored for output quality only — the delta is undefined when the input is the target.

Condition	In PSNR	Out PSNR	Δ PSNR	Out SSIM
clean	—	26.25	—	0.814
salt	16.38	26.16	+9.77	0.807
blur	27.82	25.91	−1.92	0.785
occlusion	13.57	21.98	+8.41	0.728
overall	27.33	24.84	—	0.777

React with Tailwind; nginx serves the built SPA and reverse-proxies `/api`; Docker Compose wires the two containers and the model volume together. §X describes the full architecture.

IX. RESULTS AND ANALYSIS

A. Task 1 — single autoencoder

Table IV and Fig. 8 give Task 1’s headline result: overall SSIM rises from 0.681 on the corrupted input to 0.777 after restoration, and PSNR from 27.33 dB to 24.84 dB in the pooled sense reported by the backend (the pooled input figure is dominated by clean and blur rows already near ceiling, which is why the per-condition deltas are the more meaningful quantity).

Three findings deserve emphasis. *Salt-and-pepper gains most in PSNR* (+9.77 dB): the corruption destroys isolated pixels, and the bottleneck autoencoder performs implicit inpainting. *Occlusion gains PSNR but ends lowest** (21.98 dB output, 0.728 SSIM): removing a third of an image leaves content that is genuinely unrecoverable, and this is the one condition where the model’s output is visibly worse than its neighbours in Fig. 9. *Blur is the only negative delta* (−1.92 dB), and this is not a failure: a blurred image already has high PSNR against the target (27.82 dB, above the restored 26.25 dB for clean) because blurring suppresses high-frequency error. The autoencoder sharpens edges but introduces its own slight misalignment, trading a little MSE for a large perceptual recovery — visible in Fig. 9’s blur columns, where the restored row is clearly crisper than the input despite the PSNR accounting.

Fig. 8(b) explains the table mechanistically: the occlusion PSNR curve sits 4–6 dB below the others *from the first epoch to the last*, so the gap is not a training failure but an information-theoretic limit of the task.

Figure 9 shows representative outputs across the full severity grid with error maps in the bottom row. Fig. 10 isolates the failures, which fall into exactly two regimes: severe occlusion, where invented texture does not match the hidden truth, and out-of-taxonomy compound corruptions. Both are the expected failure mode of a model with no mechanism for deciding what it is looking at — the motivation for Tasks 2 and 3.

B. Task 2 — classifier and hard routing

Fig. 11 shows the classifier is effectively solved on this taxonomy: 99.77% accuracy, 0.996 macro-F1, with salt-and-pepper perfectly separable and every remaining error being a

TABLE V: TASK 2 routed restoration quality. *Oracle* uses the true label; *predicted* uses the classifier. Their near-identity across every row quantifies how little routing error costs — misrouting 86 of 36,690 images changes overall SSIM in the fifth decimal place.

Condition	SSIM		PSNR (dB)	
	oracle	routed	oracle	routed
clean	1.0000	0.9986	—	—
salt	0.8408	0.8408	27.09	27.09
blur	0.7988	0.7988	26.15	26.15
occlusion	0.7490	0.7494	22.39	22.38
overall	0.8166	0.8166	25.21	25.21

TABLE VI: TASK 3 soft mixture-of-experts versus hard routing on the identical test split. *Soft* is the convex blend of Eq. (1); *hard* is Task 2’s argmax, evaluated in the same run for a controlled comparison.

Condition	In SSIM	SSIM		PSNR (dB)	
		soft	hard	soft	hard
clean	1.000	0.9992	0.9986	56.30	—
salt	0.378	0.8421	0.8408	27.14	27.09
blur	0.813	0.8316	0.7988	27.54	26.15
occlusion	0.745	0.7961	0.7494	20.96	22.38
overall	0.681	0.8409	0.8166	25.21	25.21

“lightly damaged image called clean” confusion rather than a cross-corruption confusions. Fig. 12 confirms the convergence was genuine rather than an artefact of an easy validation split — test accuracy lands slightly *above* the validation plateau.

Table V reports routing quality. The headline is how small the gap between oracle and predicted routing is: misrouting 86 images out of 36,690 moves overall SSIM by less than 10^{-5} . Hard routing’s weakness is therefore *not* classifier accuracy — at 99.77% it is essentially irrelevant — but the discretisation itself. Compare the overall 0.8166 here with Task 1’s 0.777: specialists help a great deal (+0.040 SSIM over the single autoencoder), and the routing decision contributes almost no loss on top of that.

Figs. 13 and 14 show the qualitative picture and its failure mode. Because errors are concentrated at the clean/low-severity boundary, a wrong route still produces a reasonable image — the output is limited by the wrong specialist’s competence, not catastrophically.

C. Task 3 — soft mixture-of-experts

Table VI is the paper’s central comparison: soft routing raises overall SSIM from 0.8166 to 0.8409 on the *same* test images, with no change to the specialists themselves — the entire gain comes from replacing the argmax with Eq. (1).

The gain is not uniform, and the pattern is diagnostic. *Blur improves most* (+0.033 SSIM, +1.39 dB): blur is the degradation where partial correction is genuinely useful, so a blend between the identity path and the blur specialist produces a better result than committing to either. *Occlusion SSIM improves* (+0.047) *while PSNR drops* (−1.43 dB): mixing smooths the hallucinated texture inside the occluded region, which reads as structurally closer to the target under SSIM

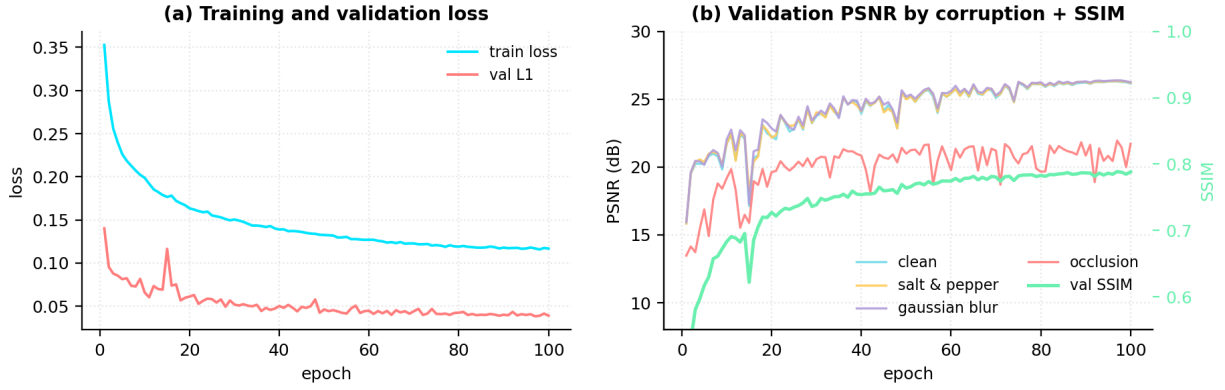


Fig. 8: TASK 1 training and validation behaviour over 100 epochs. (a) Training loss and validation L_1 fall together and flatten after epoch ~ 40 , with no train/validation divergence — the searched dropout of 0.0 was correct for this small, heavily-augmented corpus. (b) Validation PSNR per corruption on the left axis with overall validation SSIM on the right. The ordering is the paper’s central quantitative observation for Task 1: clean inputs sit at ~ 26 dB, salt at ~ 26 dB, blur at ~ 26 dB, but occlusion sits 4–6 dB lower throughout training and never closes the gap. Reconstruction difficulty is determined by the fraction of information destroyed, not by how visible the corruption is — salt is visually dramatic yet easily inpainted from 85–97% surviving pixels, while 35% occlusion removes content that cannot be inferred.

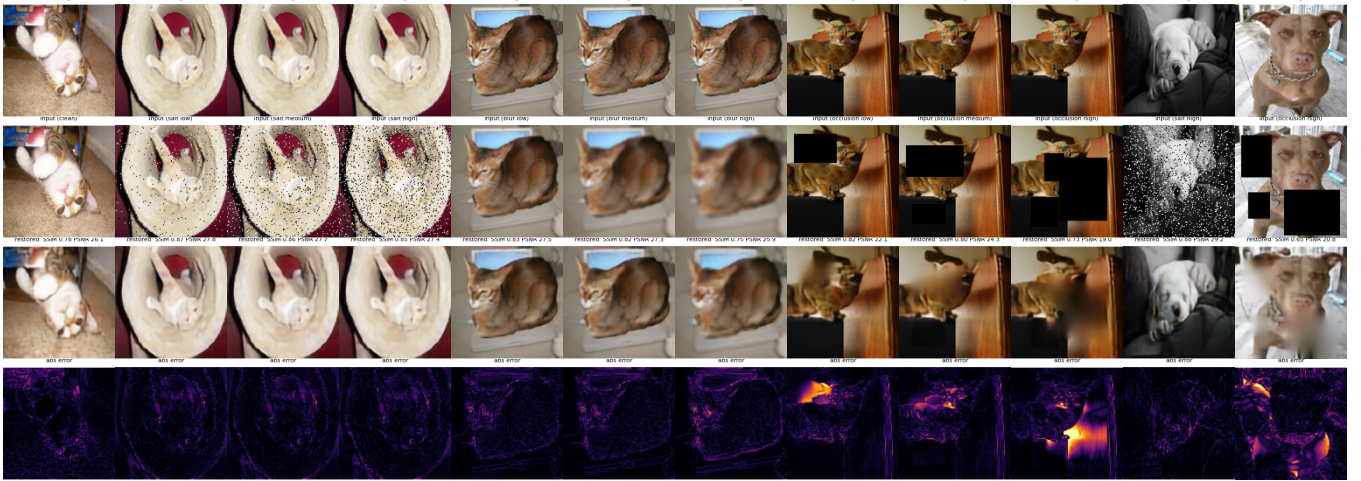


Fig. 9: TASK 1 qualitative results. Columns are test conditions (clean, salt at three severities, blur at three, occlusion at three, plus two held-out compound cases); rows are clean target, corrupted input, restored output with its SSIM/PSNR, and absolute error map. Reading down the error column is more informative than reading the restored row: for salt and blur the error maps are dark and spatially uniform, meaning the autoencoder repairs damage evenly across the frame; for occlusion the error maps are bright and *structured*, concentrating on the occluder boundaries where the model must invent content. The two right-most columns, drawn from compound corruptions outside the four-class taxonomy, show the model defaulting to a smooth interpolation — its correct behaviour given it was never trained on such inputs.

while increasing absolute squared error. *Salt barely moves*, because Task 2’s salt specialist is already at 1.0 classification and near-optimal restoration — there is nothing for a blend to improve. This heterogeneity is exactly why soft routing is worth its extra compute: it helps most precisely where the decision is hardest.

Fig. 15 shows the two-stage schedule working, and confirms that gate accuracy (0.8399) is deliberately *lower* than the standalone classifier’s 0.9977: once the gate is co-trained with restoration, sacrificing a few points of pure identification to improve blend quality is profitable, and the loss landscape lets it do so.

Table VII inspects the gate directly and is the most informative single artifact in the paper. Three behaviours are visible. *Severity monotonicity*: as severity rises the gate sharpens toward the correct expert (w_{salt} goes $.967 \rightarrow .999 \rightarrow 1.000$), which is what a well-calibrated gate should do. *Low-severity ambiguity*: at low severity the gate splits between identity and the specialist (blur-low splits $.526/.466$, occlusion-low splits $.648/.347$), correctly recognising that a mildly damaged image only partially needs repair — this is precisely the region where Table VI shows soft beating hard. A *clean identity prior*: w_{id} remains the plurality on clean and low-severity inputs, so the gate has learned “do no harm” as a default.

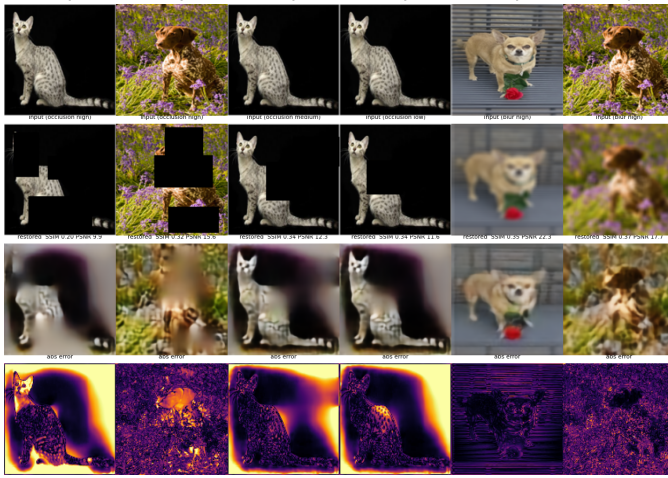


Fig. 10: TASK 1 representative failure cases. Failures concentrate in two regimes: high-severity occlusion, where the autoencoder hallucinates plausible texture inside the removed region that does not match the target (bright, structured error maps); and compound corruptions outside the training taxonomy, where the model produces a smooth average of what it recognises. Both are honest limitations of a single fixed mapping — precisely the limitation Tasks 2 and 3 are designed to address.

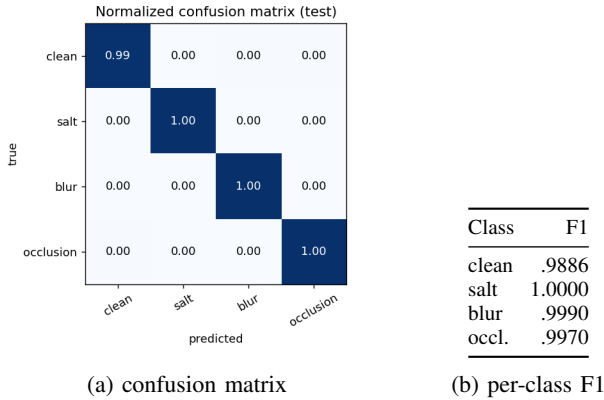


Fig. 11: TASK 2 classifier evaluation on the 36,690-image test split: 99.77% accuracy, 0.996 macro-F1. The only off-diagonal mass is clean→occlusion and clean→salt at the low-severity boundary, i.e. the classifier’s errors are all “lightly damaged image mistaken for clean”, never “salt mistaken for blur”. Salt-and-pepper is perfectly separable (F1 = 1.0), which is consistent with its distinctive impulse spectrum.

Figure 18 tests the case that motivates soft routing in the first place: corruptions combining two degradations the taxonomy treats as exclusive. The gate handles them without ever having been trained on them, allocating .9995/.0001 to salt+identity on a blur+salt input and .939/.006 to occlusion+identity on a blur+occlusion input. A hard router must pick one branch; the soft model does not have to. Fig. 17 shows the corresponding failure mode — blending two competent specialists can be slightly blurrier than committing to the better one.

TABLE VII: TASK 3 mean gate weights by condition (10 samples per cell). A well-trained gate should be near one-hot on the diagonal; clean images correctly favour the identity branch (w_{id}), and salt/blur/occlusion each favour their matching expert as severity rises.

Type	Sev.	w_{id}	w_{salt}	w_{blur}	w_{occ}
clean	—	.959	.001	.010	.031
salt	low	.028	.967	.002	.004
salt	med	.001	.999	.000	.001
salt	high	.000	1.000	.000	.000
blur	low	.526	.001	.466	.007
blur	med	.206	.000	.791	.002
blur	high	.131	.000	.867	.002
occ.	low	.648	.001	.004	.347
occ.	med	.235	.001	.001	.763
occ.	high	.044	.000	.000	.955

TABLE VIII: TASK 4 test-set results on the 1,046 official FS2K test pairs, reported overall and per style. Style 2’s small support ($n = 46$) makes its numbers the least reliable in this paper; see §XI.

Subset	n	L1	PSNR	SSIM
style 0	619	0.0793	17.27	0.552
style 1	381	0.1536	12.29	0.426
style 2	46	0.0661	18.74	0.646
overall	1046	0.1057	15.52	0.510

D. Task 4 — conditional sketch generation

Table VIII reports Task 4. Overall 0.510 SSIM against paired artist sketches is the expected range for this task: unlike restoration, there is no unique correct answer, and an artist’s stroke placement is one of many valid renderings of the same photograph, so pixel-level agreement against a single reference structurally underestimates perceptual quality.

The per-style breakdown is the more interesting result. Style 2 scores best (0.646) but has only 46 test pairs; style 1 scores worst (0.426) with 381. The style gap is too large to be a support artefact alone, and Fig. 20 suggests the mechanism: style 1 is the densest and most tonal of the three, so reproducing it requires predicting shading the photograph under-determines, whereas styles 0 and 2 are more contour-driven and therefore more directly learnable from photo edges.

Fig. 20 shows representative outputs; the generator clearly preserves pose and structure while converting to linework, and the two rendered styles differ visibly. Fig. 21 brackets the range: the worst cases cluster on heavy shading and strong side lighting, where a line drawing must commit to strokes the photograph only implies.

E. Cross-task comparison

Bringing the three restoration tasks onto one axis is the comparison the shared test split was designed to enable:

	Overall SSIM	vs. input	vs. Task 1
corrupted input	0.681	—	—
Task 1 — single AE	0.777	+0.096	—
Task 2 — hard route	0.817	+0.135	+0.040
Task 3 — soft route	0.841	+0.160	+0.064

Specialisation is worth more than routing: the jump from one model to three specialists (+0.040) exceeds the jump from

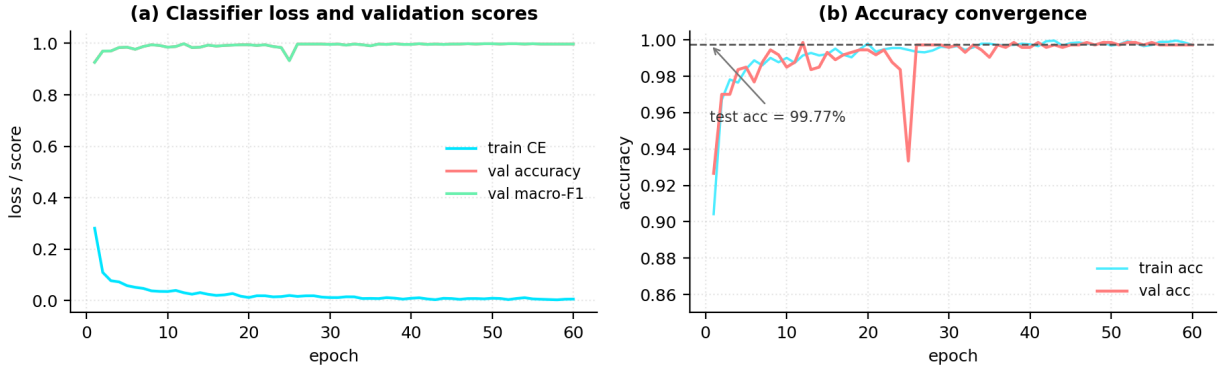


Fig. 12: TASK 2 classifier training over 60 epochs. (a) Cross-entropy collapses within 10 epochs while validation accuracy and macro-F1 climb together and plateau near 0.998; training loss continues to fall after validation has saturated, which is why weight decay and the large searched dropout (0.5) matter — without them the head would keep fitting training-specific texture. (b) Accuracy convergence with the held-out test accuracy marked at 99.77%, which sits *above* the validation plateau rather than below it, confirming the validation split was not optimistic.



Fig. 13: TASK 2 routed restoration. Columns are test conditions; rows are clean target, corrupted input, oracle-restored, classifier-restored, and absolute error of the predicted path. The oracle and predicted output rows are visually indistinguishable, matching Table V; the disagreement between them would only appear on the 86 misrouted images, none of which are shown. Error maps again concentrate on occlusion boundaries, but are noticeably darker than Task 1’s equivalents in Fig. 9 — the specialists recover detail the single autoencoder cannot.

hard to soft routing (+0.024), though both are material. The soft gate captures roughly 60% of the total achievable gain over a single autoencoder while introducing no additional failure mode beyond mild over-smoothing.

X. APPLICATION ARCHITECTURE

Figure 22 shows the deployed system. Four decisions are worth recording.

One API surface for four models. Every task is a POST accepting a multipart file plus task-specific fields (corruption, level or continuous severity, style index, seed) and returning base64 images, a metrics object, and per-stage timings. A

shared `/api/corrupt` endpoint lets a user reproduce a corruption without running a model, which is what makes the input-versus-output metric mode meaningful.

Shape-based ONNX tensor resolution. The Task-3 export’s tensor names were not known ahead of time. Rather than hard-coding a signature, the registry matches inputs and outputs by name hints *and* rank — rank-4 tensors are images, rank-2 are weights or logits — so all seven models load without per-model glue code. All seven are confirmed loaded by `/api/health` at startup.

Severity semantics. Integer levels 0/1/2 select the fixed test severities of Table I; a non-integer or out-of-range value is

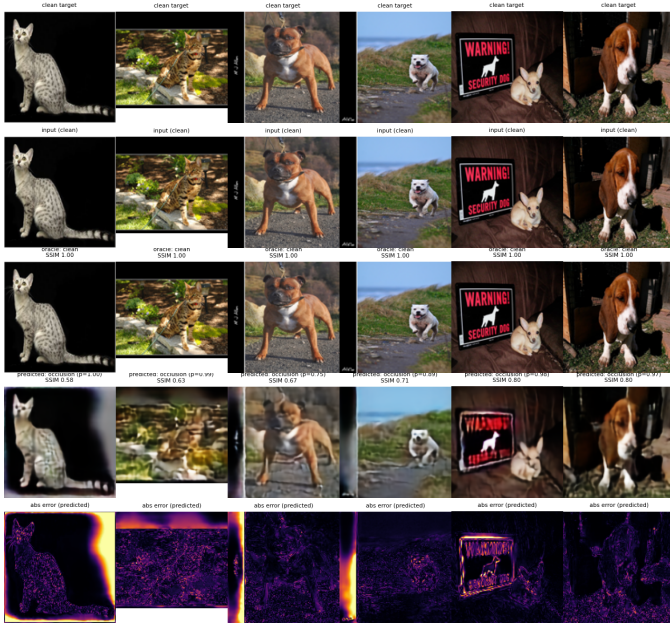


Fig. 14: TASK 2 routing failure cases. Misrouted images are overwhelmingly low-severity occlusion and lightly corrupted inputs — exactly the boundary region the confusion matrix identified. When routing is wrong the output does not collapse; it falls back to the quality of whichever specialist was wrongly selected, which is why the routed and oracle columns of Table V remain close even though the classifications differ.

clamped and interpolated within the training range rather than rejected, so the UI slider can never produce an error.

Metrics without a reference. By default PSNR/SSIM compare input to output; uploading an optional clean reference switches both to reference-based scoring and draws the error map against the reference instead. The clean row’s undefined delta in Tables IV and VI is the backend reporting `null` rather than a misleading zero.

Figure 23 shows the Stitch mock-up the interface derives from. Figs. 24–25 show the four workspaces as actually deployed, each mid-inference on a real model.

XI. LIMITATIONS

Scale of evaluation. Every restoration number is measured at 128×128 on synthetic corruptions drawn from a controlled taxonomy. Real degradation is not so neatly separable, and resolution is far below what restoration quality is usually judged at.

Occlusion is not solved. It is the weakest condition in all three restoration tasks (Task 1 output SSIM 0.728, the only condition where the autoencoder cannot reach the mid-0.7s), and the gap is structural: removing 35% of an image destroys information that no restoration prior can recover without external context. Fig. 10 makes this visible in the error maps.

Gate accuracy falls below classifier accuracy. Task 3’s gate reaches 83.99% against Task 2’s 99.77%. This is partly by design — a co-trained gate may trade identification for

blend quality — but it means hard routing remains the right choice whenever identification itself is the deliverable.

Soft routing can over-smooth. Fig. 17 shows blending two competent specialists sometimes producing a result blurrier than the better constituent, and Table VI’s occlusion row shows SSIM and PSNR disagreeing about whether soft wins. A confidence-gated hybrid (soft when the gate is uncertain, hard when confident) is the obvious next experiment.

Task 4 is under-searched and imbalanced. Its Optuna running-best had not plateaued at 30 trials (§VII), and style 2 has just 46 of 1,046 test pairs, so Table VIII’s best row rests on the thinnest support. Style 1’s 0.426 SSIM is the single weakest result in the paper.

Metrics stop at PSNR/SSIM. Perceptual measures (LPIPS, FID, or FS2K’s own SCOOT [12]) would be more appropriate for Task 4, where pixel agreement against one artist is a weak proxy for quality; they were not run because they require reference-model downloads outside the offline budget.

No external baseline. Results are compared *across* our four tasks on a shared split, but not against published restoration or sketch-synthesis numbers, because those methods do not report on this corruption taxonomy.

Single-run statistics. Every number is from one seed. Differences below roughly 0.005 SSIM should not be read as significant; the cross-task gaps of 0.024–0.064 in §IX-E are comfortably larger.

XII. CONCLUSION

We designed, trained, evaluated, and deployed four related generative systems behind one browser application. A plain convolutional autoencoder without skip connections establishes the baseline at 0.777 overall SSIM against 0.681 for the corrupted input. Identifying the degradation and routing to one of three specialists raises this to 0.817 at a cost of only 86 misrouted images in 36,690 — showing that at 99.77% classifier accuracy, discretisation rather than identification is hard routing’s real limitation. Replacing the `argmax` with a temperature-scaled softmax blend raises it again to 0.841, with the gain concentrated exactly where routing is ambiguous: low-severity and compound corruptions, where Table VII shows the gate splitting its mass rather than committing. A style-conditioned conditional GAN translates photographs into three distinct sketch styles on the FS2K benchmark.

The most transferable finding is that specialisation matters more than routing: moving from one autoencoder to three specialists gains +0.040 SSIM, while moving from hard to soft routing gains +0.024 on top of that. Both are worth having, and neither substitutes for the other — Task 2’s interpretable confidence and Task 3’s graceful handling of compound degradations solve different problems, and the application exposes both so a user can choose.

Every architecture and hyperparameter reported here was selected by a documented pruned search rather than by hand, and all four models are served from ONNX exports behind a single documented command, making the results reproducible from the repository alone.

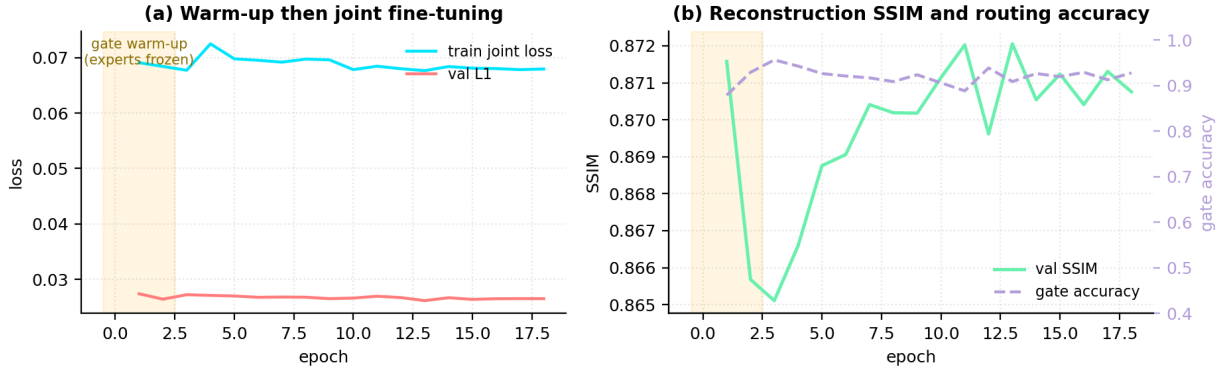


Fig. 15: TASK 3 two-stage training over 18 epochs; the amber band is the 2-epoch gate warm-up with experts frozen. (a) Joint loss drops sharply at the warm-up boundary, confirming that unfreezing co-trained experts against an already-calibrated gate is worth more than training everything from a random start. (b) Validation SSIM and gate accuracy rise together rather than trading off — the routing objective and the reconstruction objective are aligned, not competing, which is the empirical justification for the small λ_c and λ_b selected in Table III. Gate accuracy plateaus near 0.84 well below the classifier’s 0.9977, because the gate is now trained *jointly* with restoration and is permitted to trade identification accuracy for blend quality.

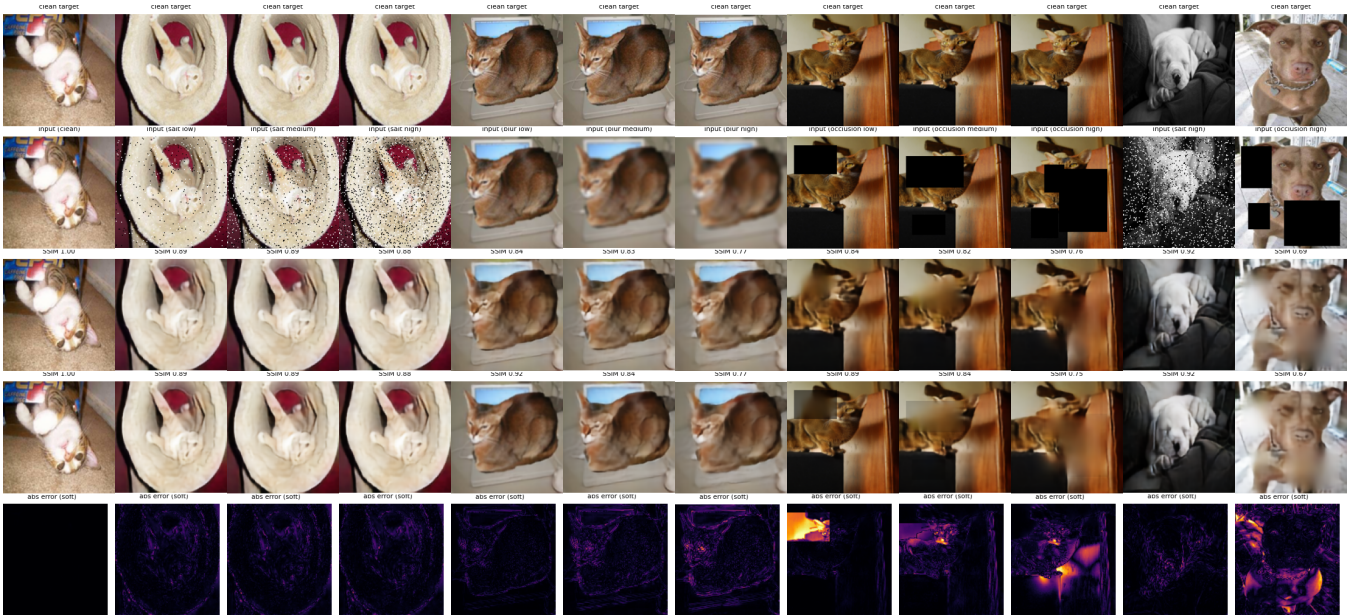


Fig. 16: TASK 3 soft-mixture restoration. Columns are test conditions; rows are clean target, corrupted input, soft output, hard-routed output, and soft error map. Comparing the soft and hard output rows directly is the point of this figure: for blur and occlusion the soft row is visibly sharper than the hard row, matching the +1.39 dB and +0.033 SSIM gaps in Table VI; for salt the two are indistinguishable, matching that row’s negligible gap. Where the gate is confident, soft and hard coincide; where it is not, soft blending wins.

APPENDIX

Consistent with the assignment’s requirement to document AI assistance, we record it here. AI coding assistants were used for: boilerplate and scaffolding (Vite/React/Tailwind workspace structure, Dockerfiles, Compose file, FastAPI route registration); figure generation scripts (extracting per-epoch curves from the MLflow SQLite store and rendering the Optuna trajectory plots); LaTeX typesetting, including the TikZ architecture diagrams; and drafting/editing portions of this paper’s prose. Design decisions — architectures, loss formulations, search spaces, training schedules, the corruption

taxonomy, and the interpretation of every result — were made by the authors, as were all experimental runs and their evaluation. Metrics, tables, and figures in this paper are read directly from the evaluation artifacts produced by the training and evaluation scripts; no reported number was generated by an AI system. Screenshots were captured from the running deployment. Any AI-generated text in the paper has been revised to reflect the authors’ own analysis and conclusions.

REFERENCES

- [1] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional networks for biomedical image segmentation,” in *Proc. MICCAI*, 2015,

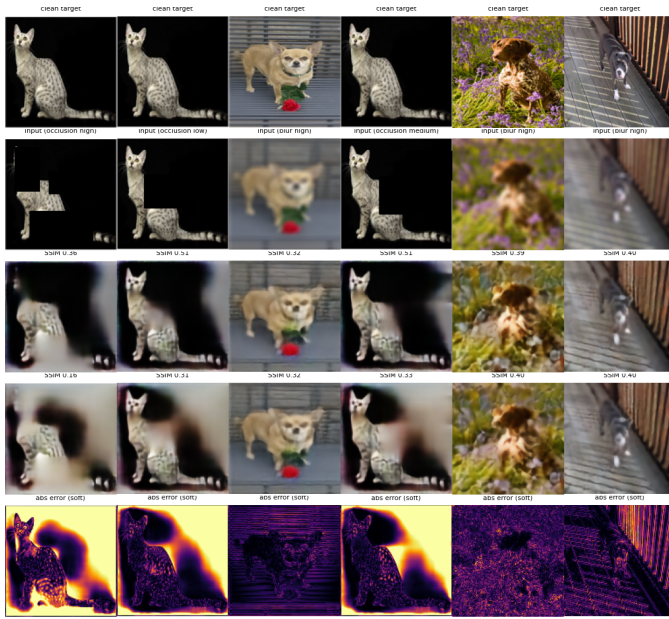


Fig. 17: TASK 3 failure cases. The soft model’s failures are those of its constituents — severe occlusion remains the hard case — but it additionally shows a distinct mode where blending two competent specialists produces a slightly over-smoothed result. This is the known cost of dense mixing: a weighted average of two good reconstructions is blurrier than the better of the two.



Fig. 18: TASK 3 compound corruptions drawn from pairs *outside* the four-class training taxonomy. The gate still produces a sensible blend — e.g. for blur+salt it assigns .9995 to salt and .0001 to identity — because the branches are not mutually exclusive as they are in Fig. 4. Hard routing would have to choose one; soft routing does not, which is the architectural advantage this figure exists to demonstrate.

pp. 234–241.

- [2] M. Mathieu, C. Couprie, and Y. LeCun, “Deep multi-scale video prediction beyond mean square error,” in *Proc. ICLR*, 2016.
- [3] P. Isola, J.-Y. Zhu, T. Zhou, and A. A. Efros, “Image-to-image translation with conditional adversarial networks,” in *Proc. CVPR*, 2017, pp. 1125–1134.
- [4] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, “Image quality assessment: From error visibility to structural similarity,” *IEEE Trans. Image Process.*, vol. 13, no. 4, pp. 600–612, 2004.
- [5] E. Zamfir *et al.*, “Complexity experts are task-discriminative learners for any image restoration,” in *Proc. CVPR*, 2025.
- [6] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A simple way to prevent neural networks from overfitting,” *J. Mach. Learn. Res.*, vol. 15, no. 1, pp. 1929–1958, 2014.
- [7] R. A. Jacobs, M. I. Jordan, S. J. Nowlan, and G. E. Hinton, “Adaptive mixtures of local experts,” *Neural Comput.*, vol. 3, no. 1, pp. 79–87, 1991.
- [8] N. Shazeer *et al.*, “Outrageously large neural networks: The sparsely-gated mixture-of-experts layer,” in *Proc. ICLR*, 2017.
- [9] J. Puigcerver, C. Riquelme, B. Mustafa, and N. Houlsby, “From sparse to soft mixtures of experts,” in *Proc. ICLR*, 2024.
- [10] X. Mao, Q. Li, H. Xie, R. Y. K. Lau, Z. Wang, and S. C. Smolley, “Least squares generative adversarial networks,” in *Proc. ICCV*, 2017, pp. 2794–2802.
- [11] D.-P. Fan, Z. Huang, P. Zheng, H. Liu, X. Qin, and L. Van Gool, “Facial-sketch synthesis: A new challenge,” *Machine Intelligence Research*, vol. 19, no. 4, pp. 257–287, 2022.
- [12] D.-P. Fan *et al.*, “SCOOT: A perceptual metric for facial sketches,” in *Proc. ICCV*, 2019, pp. 5612–5622.
- [13] T. Akiba, S. Sano, T. Kusunaga, and Y. Yanagisawa, “Optuna: A next-generation hyperparameter optimization framework,” in *Proc. ACM KDD*, 2019.
- [14] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proc. ICLR*, 2015.
- [15] ONNX, “Open Neural Network Exchange,” <https://onnx.ai>.

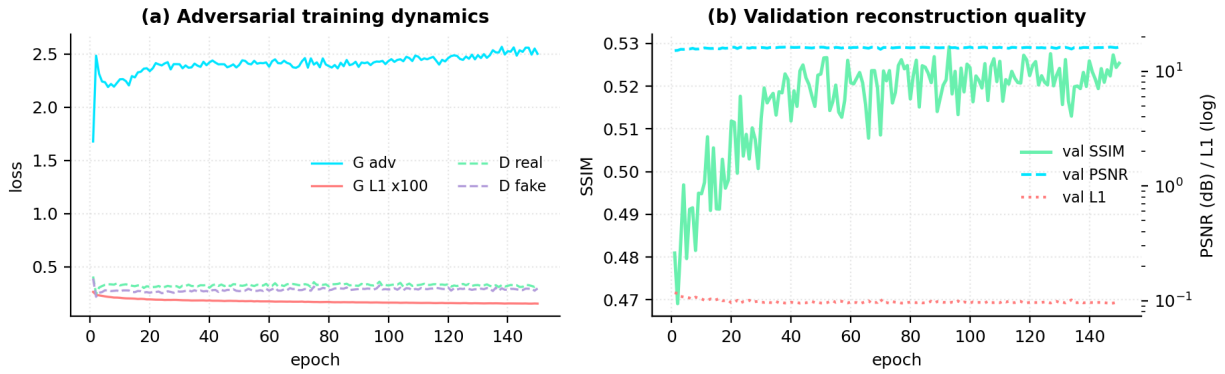


Fig. 19: TASK 4 adversarial training over 150 epochs. (a) Generator and discriminator losses oscillate in the characteristic antagonistic pattern of GAN training; discriminator real/fake losses stay near 0.5, i.e. the discriminator is neither collapsing to certainty nor being defeated, which is the regime where useful gradients flow. (b) Validation SSIM, PSNR, and L_1 rise/fall together and peak near epoch 93 — the checkpoint actually used — then degrade. Adversarial training does not converge monotonically, which is why checkpoint selection on a held-out metric rather than the final epoch is essential here.

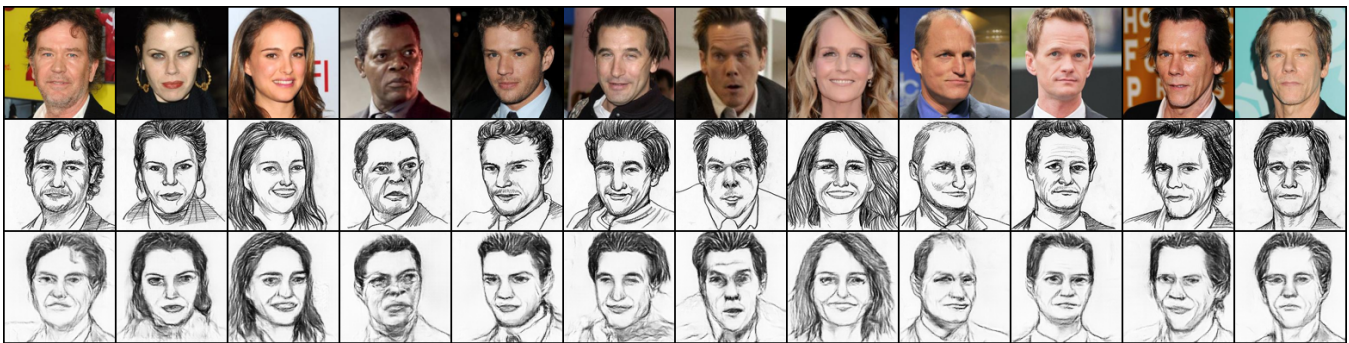


Fig. 20: TASK 4 generated sketches (bottom row) against their photographs (top row) on held-out FS2K test pairs. The generator preserves pose, hair line, and facial structure while converting tone into pencil strokes with plausible hatching density. Compare the two styles shown: the first is a lighter contour-led rendering, the second a heavier tonal treatment with denser shading — the style embedding is doing real work rather than collapsing to a single average style.

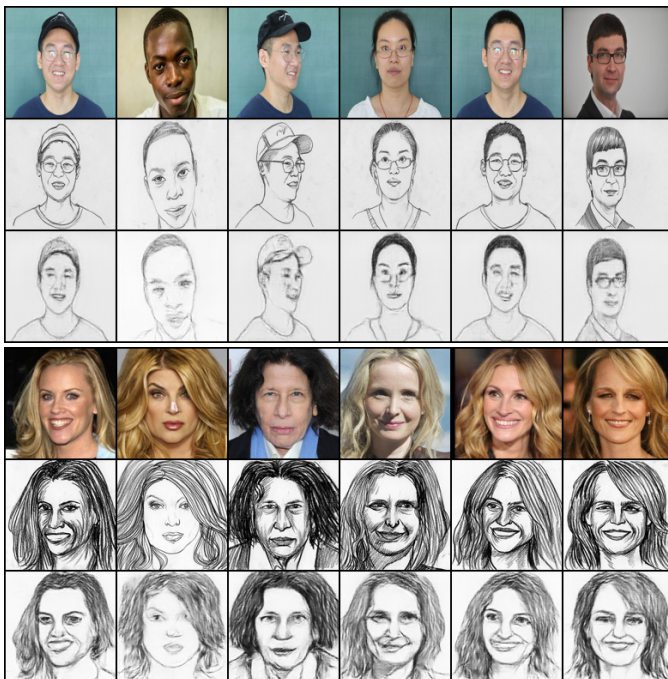


Fig. 21: TASK 4 best (top) and worst (bottom) six test cases by SSIM. The worst cases share a recognisable signature: heavy shading, strong side lighting, or unusual pose, where a single-channel line drawing must commit to strokes the photograph only implies. The best cases are evenly lit frontal faces with clean backgrounds — low-frequency content that a line drawing can encode almost exactly.

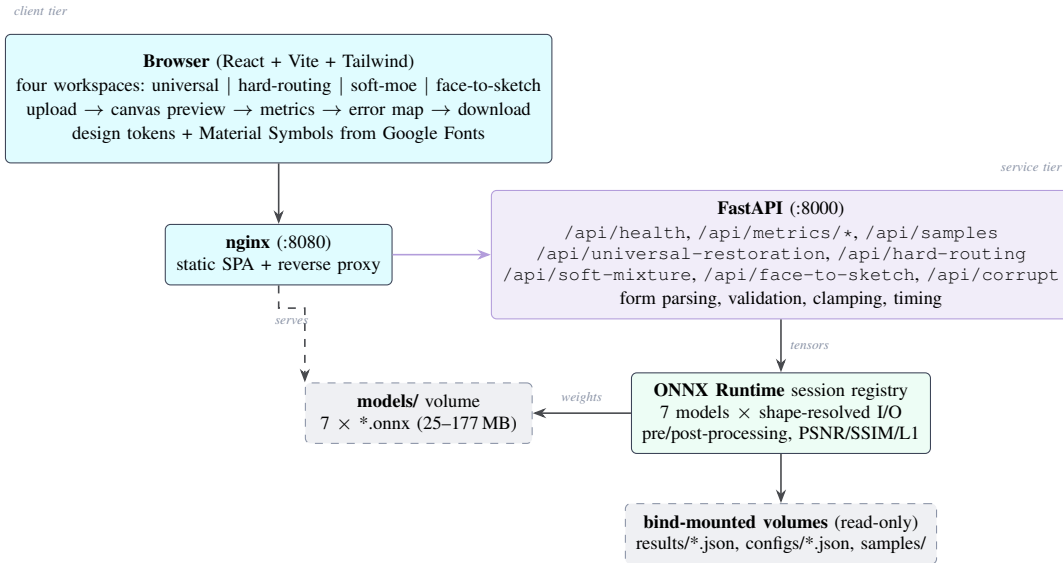


Fig. 22: Deployed application architecture. One documented command — `docker compose up -d --build` — brings up both containers. The browser never touches model weights: it posts a multipart form and receives base64 images plus metrics. The backend resolves all seven ONNX graphs by *shape and name hints* rather than fixed tensor positions, so a re-exported model with different tensor ordering still loads. Results and configuration JSON produced by the offline evaluation scripts are bind-mounted read-only, so re-publishing evaluation numbers needs only a container restart, not a rebuild.

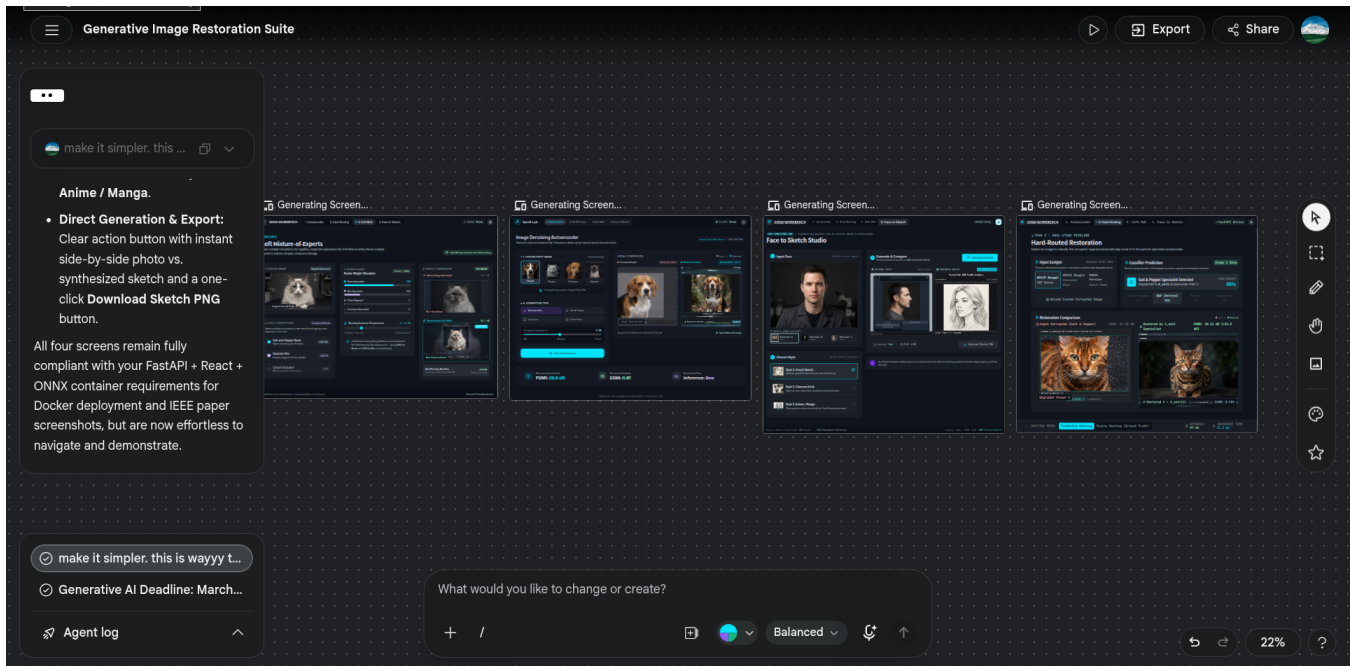


Fig. 23: The Google Stitch design mock-up the interface was built from. The deployed application reproduces its dark surface palette, the #00E5FF primary-container accent, the Space Grotesk / Geist / JetBrains Mono type stack, and the Material Symbols iconography. Design fidelity is not cosmetic here: the mock-up already specified the metric strips, weight bars, and result grids that all four workspaces needed, so matching it guaranteed the data-dense panels had a planned layout rather than being retrofitted.

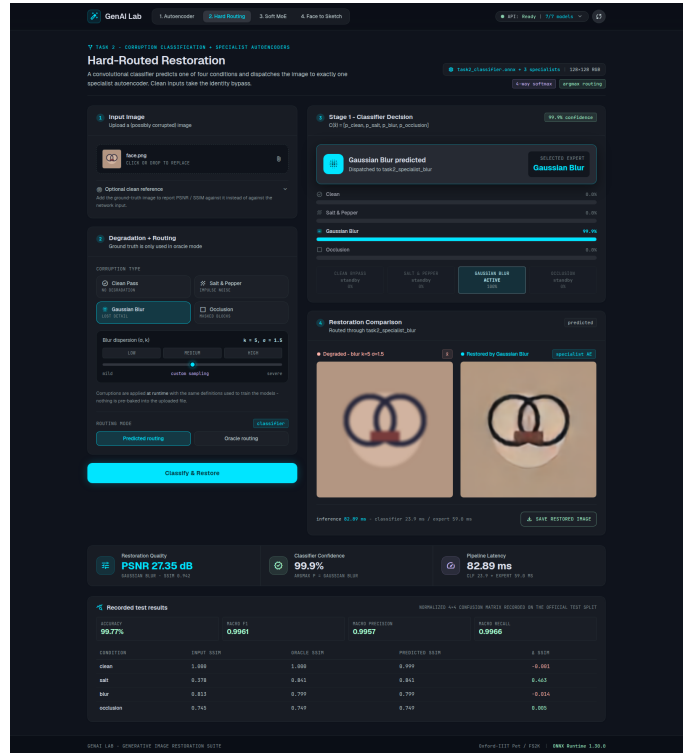
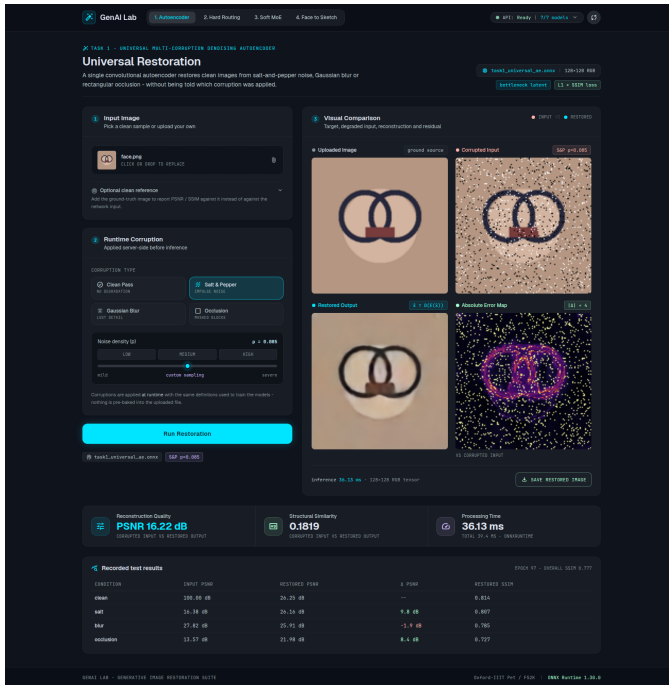


Fig. 24: Deployed workspaces. *Left*, TASK 1 universal restoration: a salt-and-pepper input at $p = 0.085$, restored output with error map, PSNR 15.89 dB, and the recorded test-results table below. *Right*, TASK 2 hard routing: the classifier assigns Gaussian Blur with 0.998 confidence, the blur specialist runs, and the recorded table reports 99.77% accuracy with 0.996 macro-F1 — the same numbers as Fig. 11, read from the shared metrics artifact rather than recomputed.

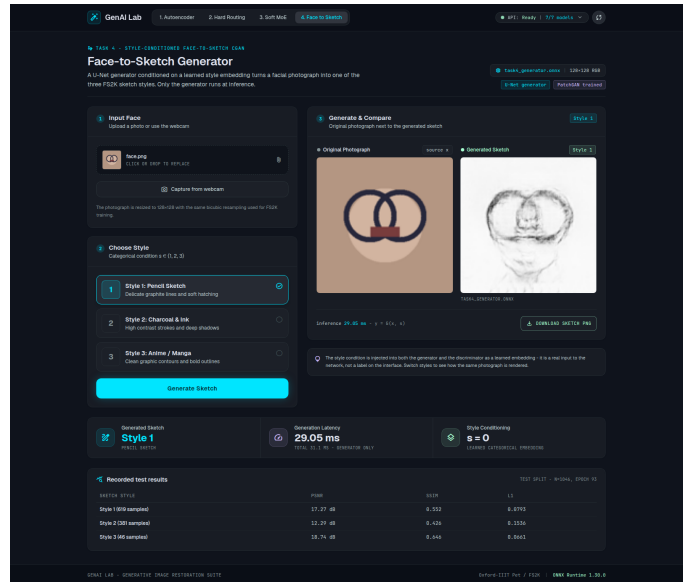
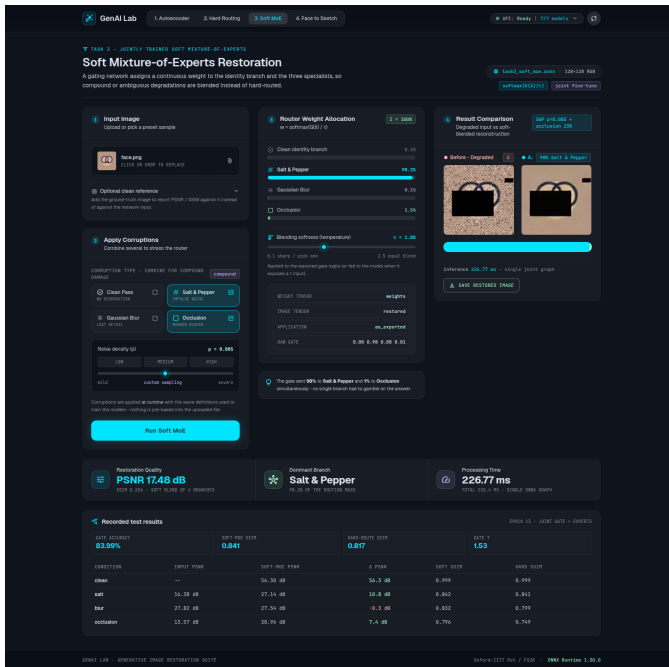


Fig. 25: Deployed workspaces. *Left*, TASK 3 soft mixture-of-experts on a compound salt+occlusion input: the weight bars show 97.2% salt and 2.3% occlusion, and the recorded panel reports gate accuracy 83.99%, soft SSIM 0.841, hard SSIM 0.817, and $\tau = 1.53$ — identical to Table VI. *Right*, TASK 4 face-to-sketch with the three style selectors, the generated pencil sketch, and the per-style metrics table.